WEB APPLICATION BASICS

TryHackMe Learning & Practical Report

Prepared by	Nethra M.
Platform	TryHackMe
Room	Web Application Basics
Learning path	Cyber Security 101
Focus area	Web Application Security / WAPT Foundations
Completion status	Completed

Report purpose

This report documents the concepts and practical learning from the TryHackMe Web Application Basics room. It explains how browsers and web servers communicate, how HTTP messages are structured, and why requests, responses, cookies, and security headers matter during web application security testing.

Prepared for personal learning documentation, portfolio use, and cybersecurity training.

1. Executive Summary

I completed the **Web Application Basics** room on TryHackMe as part of my web application security learning. The room helped me understand how a browser sends requests, how a web server responds, how URLs identify resources, and how headers and cookies carry important information.

The practical API exercises connected theory with real request methods. They demonstrated how GET can retrieve information, POST can submit or change data, and DELETE can request removal of a resource. These examples highlighted why applications must verify authorization before allowing users to access or change data.

The room also introduced defensive controls such as Content Security Policy (CSP), HTTP Strict Transport Security (HSTS), X-Content-Type-Options, and the Secure and HttpOnly cookie attributes. These controls help reduce risk when used with sound authentication, authorization, session management, and input validation.

2. Lab Information

FIELD	DETAIL
Platform	TryHackMe
Room	Web Application Basics
Learning path	Cyber Security 101
Primary domain	Web Application Security
Purpose	Foundational learning
Completion	Completed
Primary skills	HTTP, URLs, requests, responses, headers, cookies, API methods

Room link: https://tryhackme.com/room/webapplicationbasics

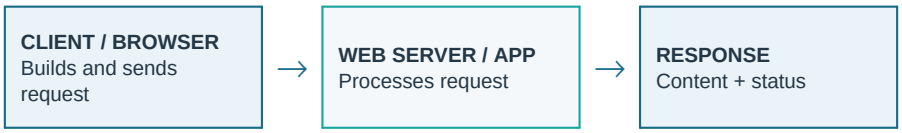
3. Learning Objectives

The main objectives of this learning activity were to:

- Identify the client, server, and communication flow in a web application.
- Recognize URL components, including scheme, hostname, path, and query string.
- Understand the structure of HTTP request and response messages.
- Recognize common request methods and status-code groups.
- Explain the purpose of headers, cookies, and browser security headers.
- Practice basic API interaction using GET, POST, and DELETE in the authorized lab.

4. Web Application Architecture Basics

A typical web application involves a client, a server, and network communication between them. The browser acts as the client: it sends a request and displays the returned content. The web server receives the request and returns a resource or application response. A Web Application Firewall (WAF) may sit in front of the application to inspect traffic and help block suspicious requests.



From a penetration-testing perspective, each layer can expose a different attack surface. A tester may examine how the browser constructs requests, how the server handles unexpected input, and whether security controls correctly restrict access. Testing should remain within systems and environments where permission has been granted.

5. URL Structure and Security Relevance

A Uniform Resource Locator (URL) identifies a resource and gives the client the information needed to request it. Example:

```
https://example.com/products?id=10
```

- **Scheme:** https — identifies the protocol used for the connection.
- **Hostname:** example.com — identifies the target host.
- **Path:** /products — identifies a resource or route.
- **Query string:** ?id=10 — carries parameters used by the application.

HTTPS protects communication in transit through encryption and server authentication. Query parameters may carry search terms, identifiers, filters, or other input. During testing, changing a parameter in an authorized lab can show how the application behaves; the server must not trust client-controlled values without validation and authorization checks.

Typosquatting uses domain names that resemble legitimate domains, often through small spelling changes. Users may mistakenly visit these domains, so checking domain names carefully is an important security habit.

6. HTTP Messages

HTTP communication uses a request from the client and a response from the server. A message can include a request line or status line, headers, a blank line, and an optional body. The body may contain form data, JSON, HTML, or other content.

```
GET /products?id=10 HTTP/1.1\nHost: example.com\nUser-Agent: Browser\n
```

Tools such as Burp Suite can be used in authorized testing to intercept, inspect, modify, and resend HTTP messages. Comparing responses after controlled changes helps a tester understand application behavior.

7. HTTP Requests: Methods and Components

HTTP methods communicate the action the client intends to perform. The room covered common methods including GET, POST, and OPTIONS, while the practical exercises also demonstrated DELETE.

METHOD	TYPICAL PURPOSE	SECURITY CONSIDERATION
GET	Retrieve a resource or information.	Return only data the requesting user is allowed to see.
POST	Submit data or trigger an operation.	Validate input and verify permission to make changes.
OPTIONS	Ask which communication options a resource supports.	Ensure exposed methods and responses are appropriate.
DELETE	Request removal of a resource.	Protect destructive actions with authorization, validation, and logging.

The request line identifies the method, target path, and HTTP version. The Host header identifies the destination host. Other headers can describe the client, accepted formats, authentication information, cookies, and submitted data. Form data may use application/x-www-form-urlencoded; JSON APIs commonly use application/json.

8. HTTP Responses and Status Codes

An HTTP response contains a status line, response headers, and—when applicable—a response body. The status code gives a quick indication of how the server handled the request.

CODE	MEANING	INTERPRETATION
200	OK	The request succeeded.
301	Moved Permanently	The resource has a permanent redirect.
403	Forbidden	The server refuses to fulfill the request.
404	Not Found	The requested resource was not found.
500	Internal Server Error	The server encountered an unexpected condition.

Status codes are useful signals during testing, but they do not prove a vulnerability by themselves. Differences among success, redirect, client-error, and server-error responses can help a tester understand endpoint behavior, access restrictions, or application errors.

9. Response Headers, Cookies, and Security

Response headers can disclose information or influence browser behavior. The Server header may reveal details about server software. Avoiding unnecessary version disclosure can reduce information available to an attacker, but it does not replace patching and secure configuration.

Cookies often maintain session state. The Secure attribute instructs the browser to send a cookie only over HTTPS. HttpOnly prevents client-side JavaScript from directly reading the cookie, reducing the impact of some XSS scenarios involving cookie theft. These attributes should be combined with sound session handling and access controls.

10. Security Headers

Security headers help browsers apply additional protections. They are defense-in-depth controls and should be configured to suit the application.

- **Content-Security-Policy (CSP):** Restricts permitted sources for scripts and other browser resources. A directive such as script-src defines where scripts may load from.
- **HTTP Strict Transport Security (HSTS):** Tells browsers to use HTTPS for a site for a specified period. includeSubDomains can extend the policy to subdomains.
- **X-Content-Type-Options: nosniff:** Helps prevent browsers from MIME-sniffing a response and interpreting it as a different content type.

A header's presence alone does not guarantee that a site is secure. The policy must be appropriate, and the application still needs secure authentication, authorization, input handling, and session management.

11. Practical Task — API Requests

The practical portion connected protocol concepts to API interactions. The lab used user-resource endpoints to demonstrate retrieving, modifying, and deleting data. The examples below document the request patterns and the security lesson from each operation.

11.1 GET Request — Retrieving Data

```
GET /api/users
```

The GET request retrieved the available user list. This demonstrates a typical read operation. In a real application, an endpoint should return only information the requesting user is authorized to access. API enumeration and excessive data exposure are important areas to review during an authorized assessment.

11.2 POST Request — Modifying Data

```
POST /api/user/2\ncountry=US
```

The POST exercise updated the country field for a user resource. It demonstrated that request bodies can carry application data and that a request may change server-side state. A secure application must validate the submitted value and check whether the authenticated user may modify that resource.

11.3 DELETE Request — Removing Data

```
DELETE /api/user/1
```

The DELETE exercise demonstrated a request to remove a user resource. Because deletion is destructive, production systems should enforce authorization and validation, record audit logs, and provide confirmation or recovery controls where suitable.

Practical note: These request examples follow the exercise patterns described in the supplied reference report. Add your own lab screenshots or completion evidence if required; no screenshot is embedded in this document.

12. Key Security Takeaways

- HTTP methods describe intended operations, but the server must independently enforce authorization.
- URL parameters and request bodies are controlled by the client and must be validated.
- Status codes and response differences help explain application behavior but are not proof of a flaw by themselves.
- Security headers and cookie attributes provide browser-side protections, but do not replace secure application design.
- API endpoints should enforce authentication, authorization, input validation, and appropriate data exposure.
- Understanding request and response messages is essential preparation for tools such as Burp Suite.

13. Skills Practiced

AREA	LEARNING GAINED
Web fundamentals	Client/server architecture and how applications deliver content.
URLs	Scheme, host, path, query strings, and security relevance.
HTTP	Request/response structure and message components.
Methods	GET, POST, OPTIONS, and DELETE use cases.
Headers and status codes	Interpreting metadata and server responses.
Cookies and security headers	Secure, HttpOnly, CSP, HSTS, and nosniff concepts.
API testing	Basic retrieval, modification, and deletion in the lab.

14. What I Would Explore Next

This room provides a foundation for deeper Web Application Penetration Testing (WAPT). My next steps are to explore authentication and session management, access control and IDOR, input validation, SQL injection, cross-site scripting, file inclusion, command injection, SSRF, API security, and business-logic vulnerabilities in authorized training labs.

I also plan to improve my use of Burp Suite by intercepting requests, changing parameters, replaying requests, comparing responses, and recording findings. These activities build directly on the request/response model studied in this room.

15. Conclusion

Completing Web Application Basics gave me a clearer understanding of how web applications communicate at the HTTP level. I learned to look beyond a website's visual interface and consider its requests, responses, resources, parameters, headers, cookies, and server-side actions.

The API exercises showed how HTTP methods can retrieve, modify, and delete application data. This knowledge will help me approach future web security labs systematically: first understand normal behavior, then assess where authorization, input handling, trust boundaries, or security controls may fail.

Completion status: Completed — TryHackMe Web Application Basics.

16. References

1. TryHackMe — Web Application Basics: https://tryhackme.com/room/webapplicationbasics

2. Report structure and room-specific practical examples were based on the user-provided reference PDF and rewritten as an original learning document. Confirm practical details against your own completed room if your instructor requires evidence.

Prepared by Nethra M. | Web Application Security Foundations